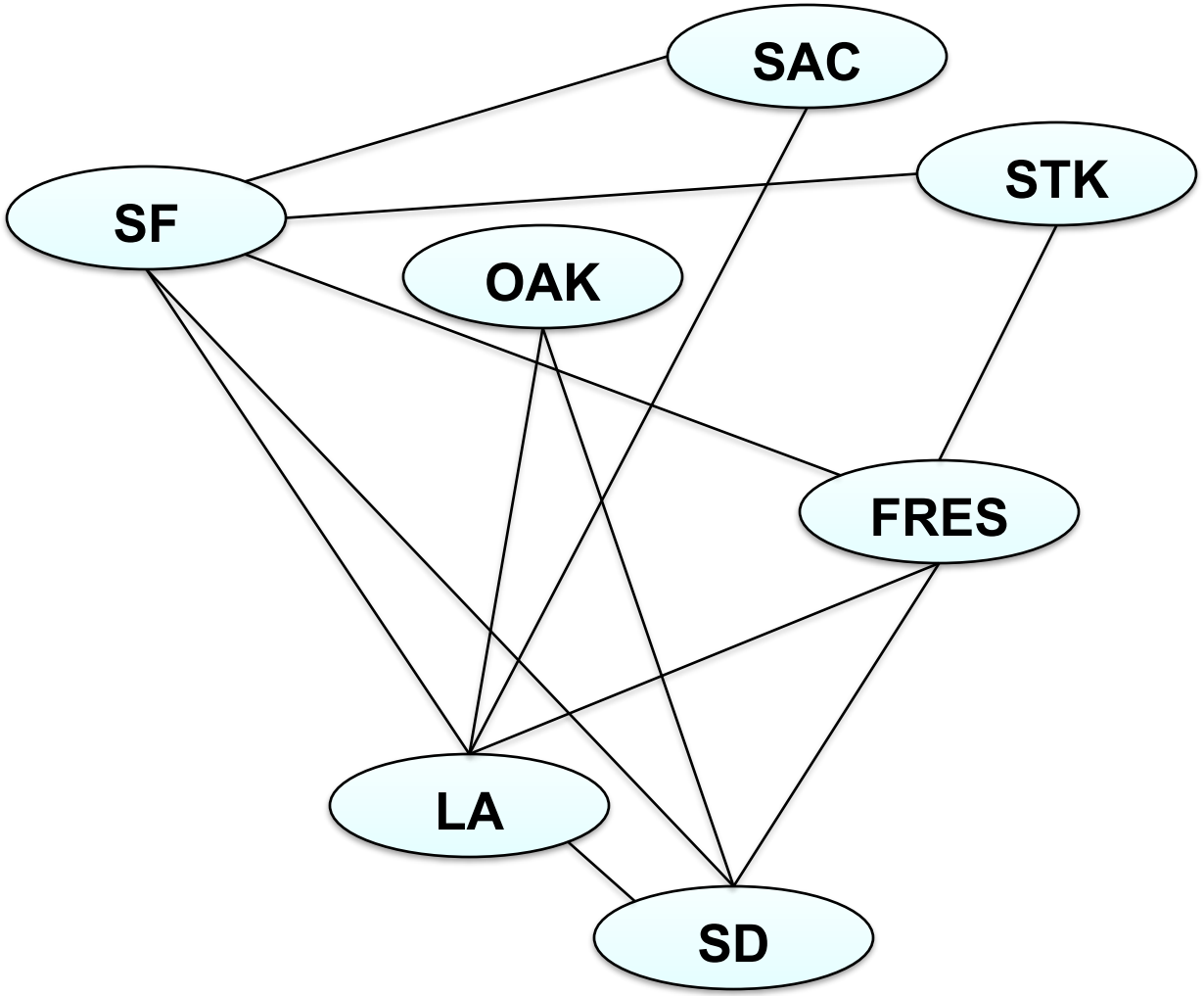
Elementary Graph Algorithms

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

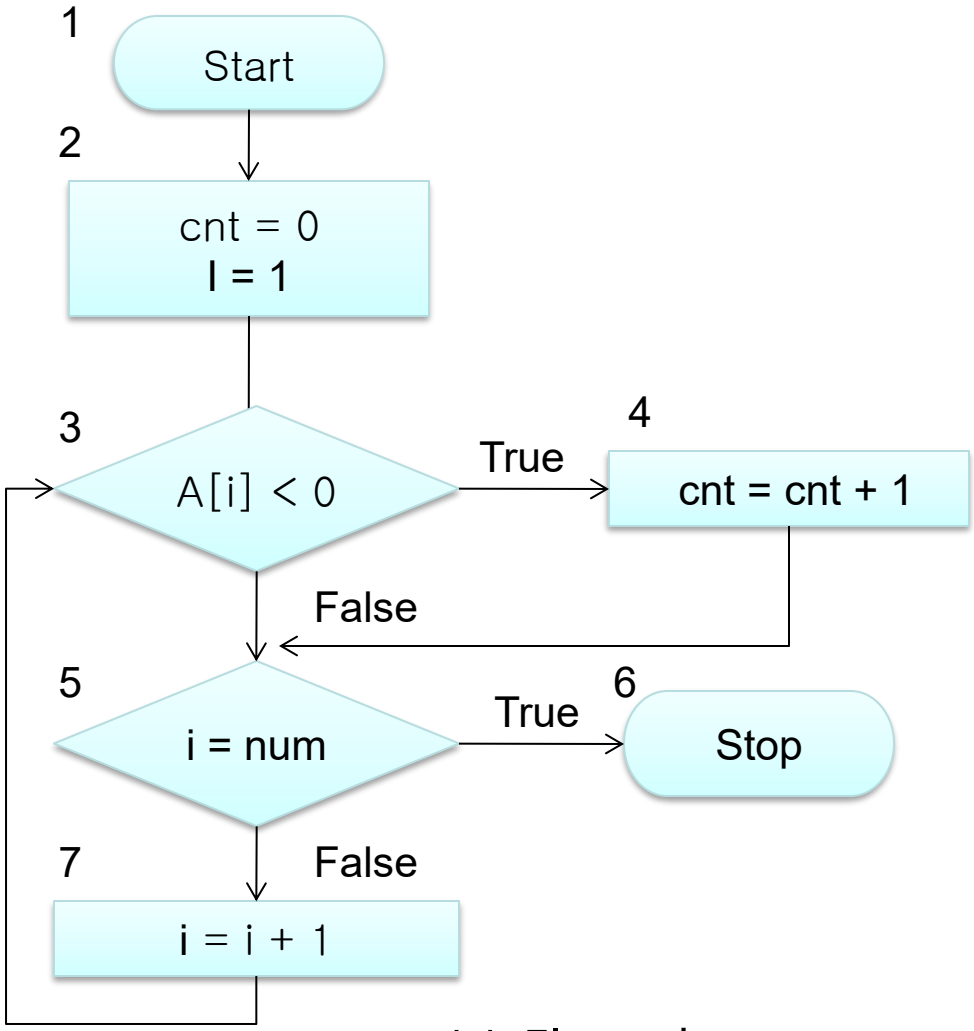
QUESTIONS…

1. What is the cheapest way to fly from Pohang to New York?
2. Which route involves the least flying time?
3. If one City's airport is closed by bad weather, can you still fly between every other pair of cities?
4. If one computer in a network goes down, can a messages be sent between every other pair of computers in the network?
5. Does a flow chart have any loop?
6. How should wires be attached to various electrical outlets so that all are connected together using the least amount of wire?

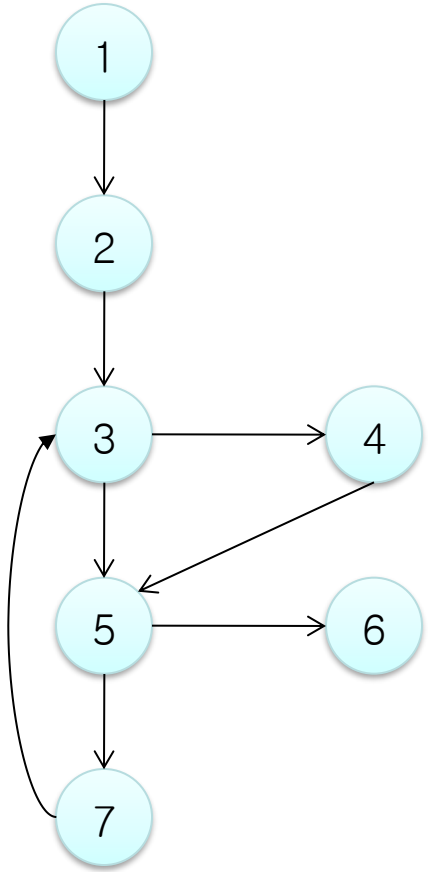
Problems: Airline Routes



PROBLEMS: FLOWCHARTS



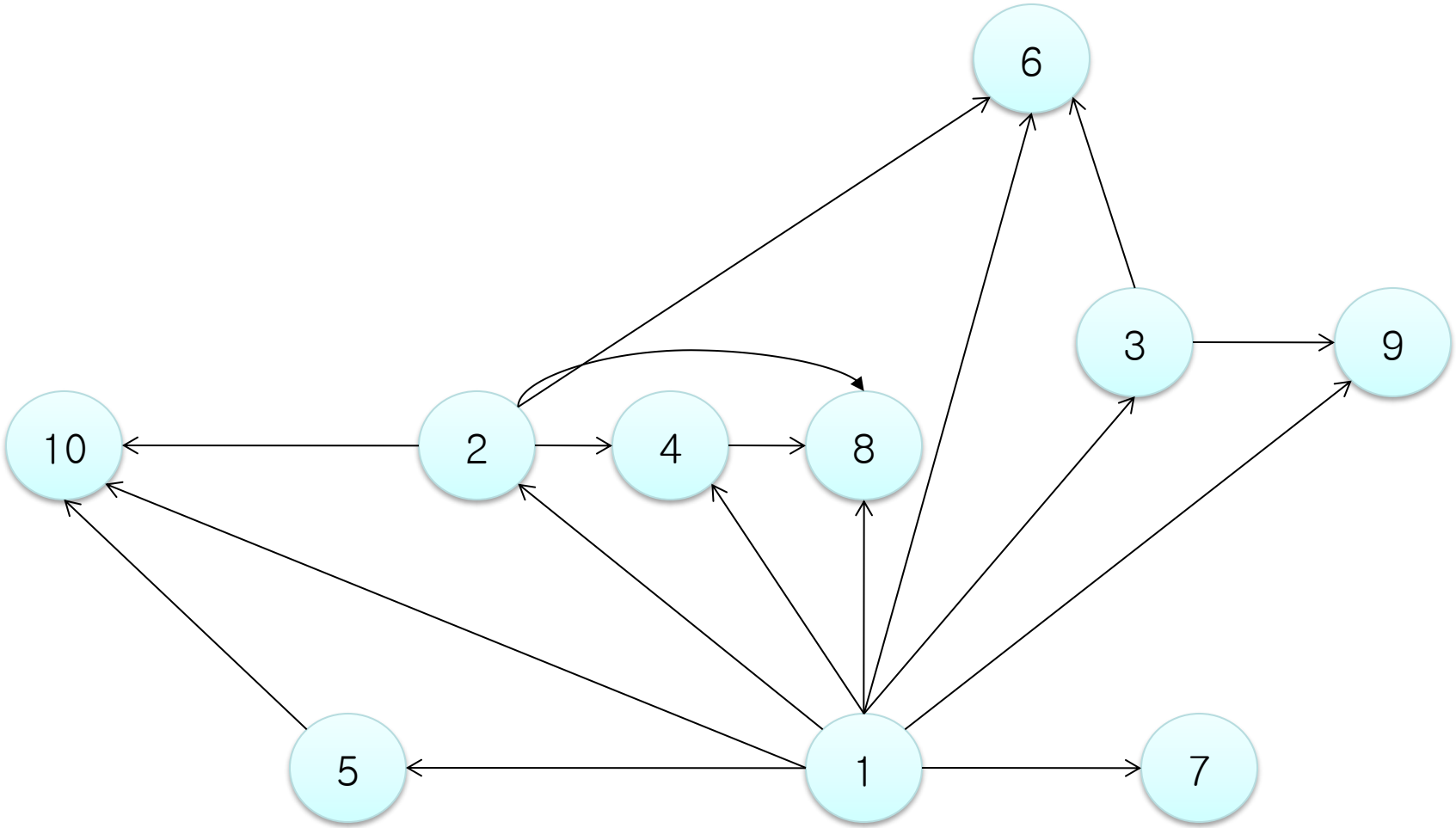
(a) Flow chart



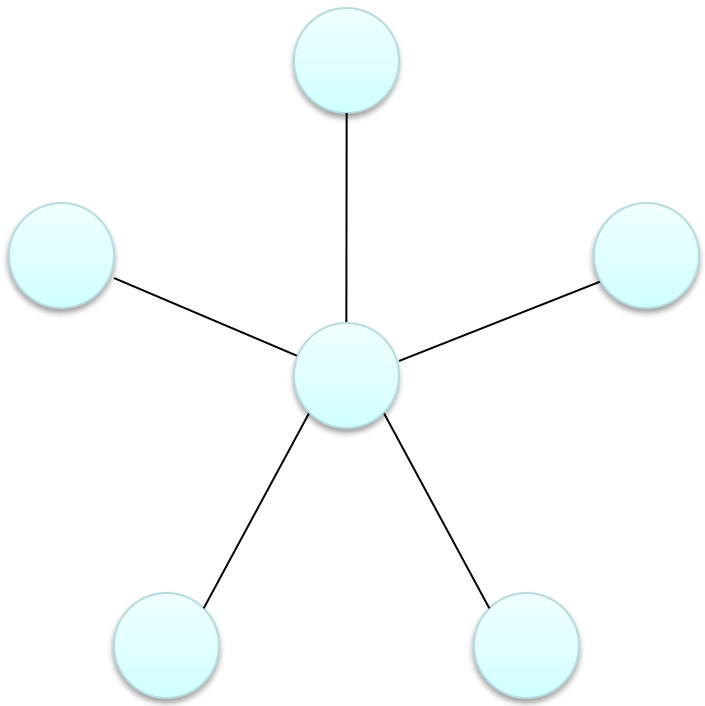
(b) Directed graph

PROBLEMS: BINARY RELATION

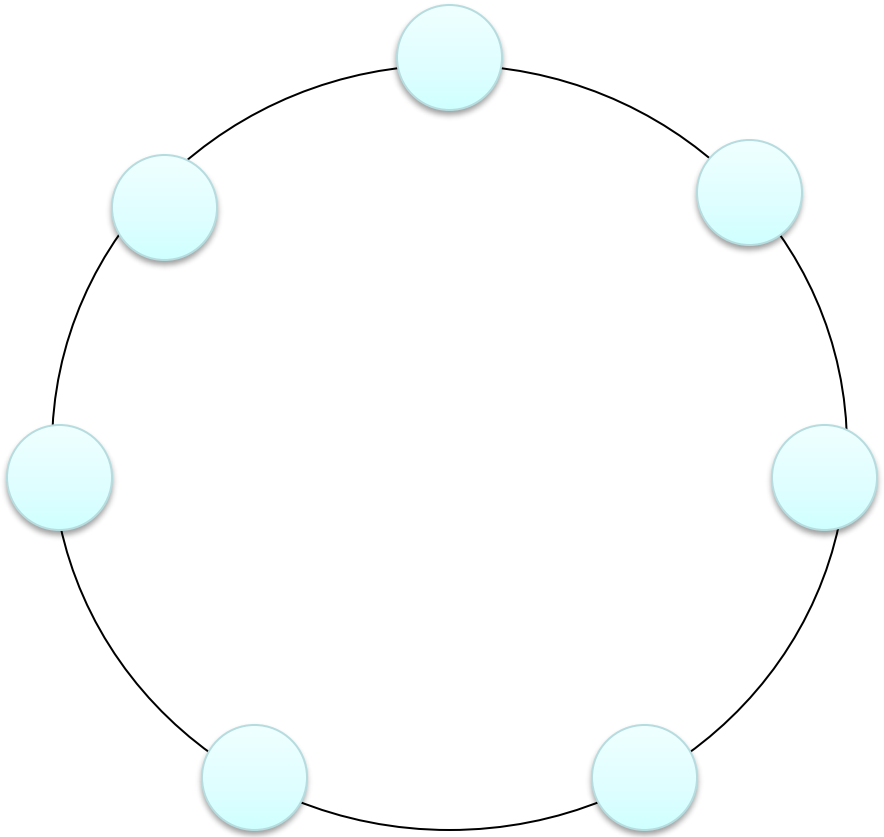
- x is a proper factor of y



Problems: Computer Networks



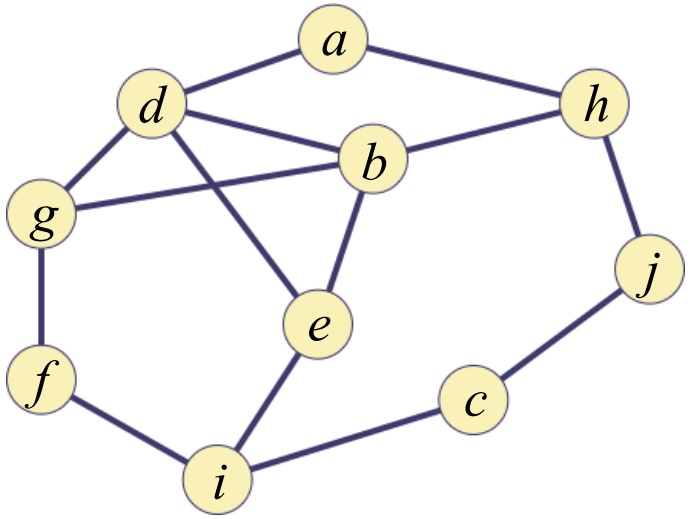
(a) A star network



(b) A ring network

Terms You Should Know or Learn Now

- **Vertex (plural vertices) or Node(s)**
  - ✓ Adjacent nodes: nodes connected by an edge
- **Edge (sometimes referred to as an arc)**
  - ✓ Note the meaning of incident
- **Degree of a vertex: how many adjacent vertices**
  - ✓ Digraph: in-degree (num. of incoming edges) vs. out-degree



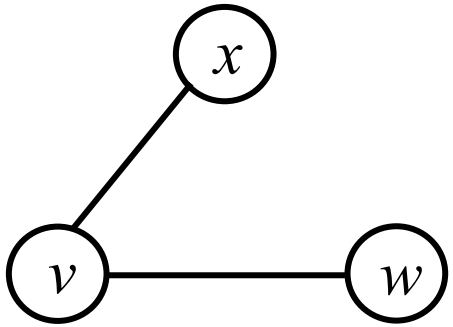
Incidence of a Vertex / Edge

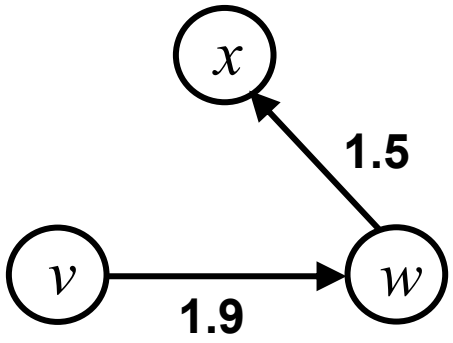
Definition: For a graph $G = (V(G), E(G))$, two vertices $x, y \in V(G)$ which are adjacent forming an edge $\{x, y\}$ are said to be **Incident** to that edge. Alternatively, the edge $\{x, y\}$ is also incident to the vertices x and y.

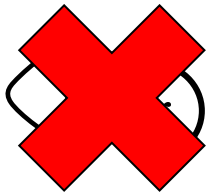
From the graph above, we can say that the edge ad is incident to vertices a and d. We can also say that vertex a is incident to edge ad, and vertex d is incident to edge ad.

Terms You Should Know or Learn Now

- **Graphs can be:**
  - ✓ Directed or undirected
  - ✓ Weighted or not weighted
    - – weights can be reals, integers, etc.
    - – weight also known as: cost, length, distance, capacity,…
- **Undirected graphs:**
  - ✓ Normally an edge can't connect a vertex to itself

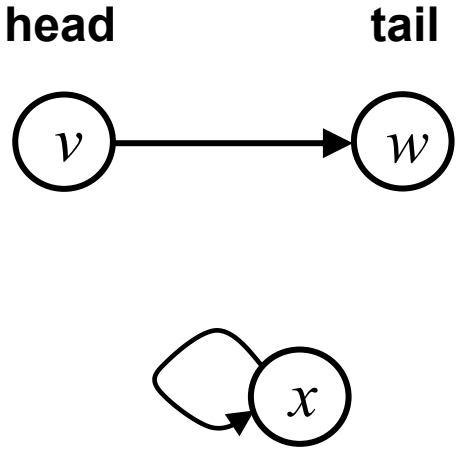


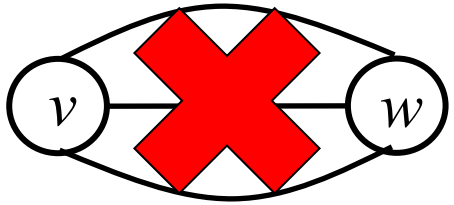




Terms You Should Know or Learn Now

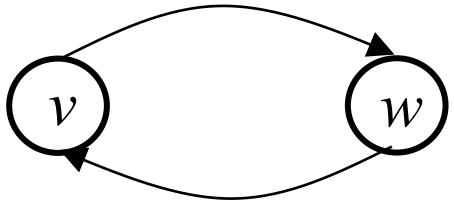
- **A directed graph (also known as a digraph)**
  - ✓ "Originating" node is the head, the target is the tail
  - ✓ An edge may (or may not) connect a vertex to itself
- **A graph may not have multiple occurrences of the same edge.**



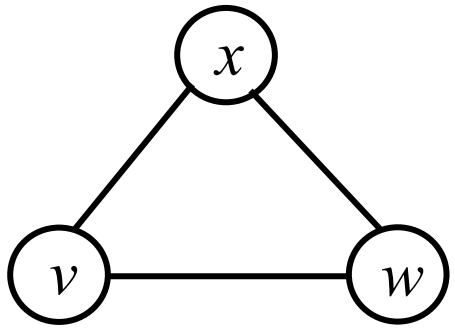


Terms You Should Know or Learn Now

- **Symmetric digraph**
  - ✓ For every edge vw, there is also the reverse edge wv.



- **Complete graphs**
  - ✓ There is an edge with each pair of vertices.



Terms You Should Know or Learn Now

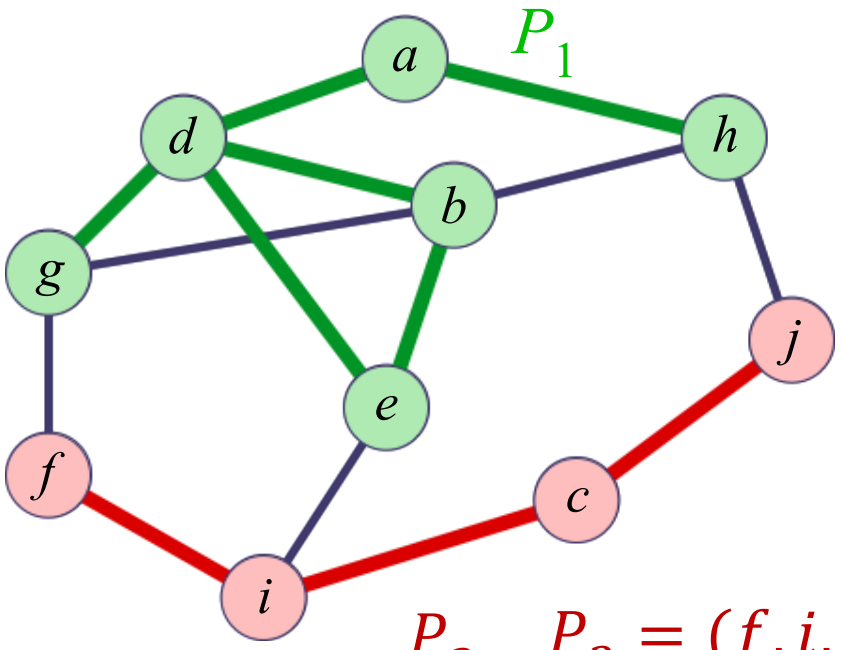
- **Size of graph? Two measures:**
  - ✓ Number of nodes. Usually denoted by n
  - ✓ Number of edges: Usually denoted by m
- **Dense graph: many edges**
  - ✓ Undirected:
    - – Each node connects to all others, so the graph with $m = n(n-1)/2$ is called a *complete graph*.
  - ✓ Directed: $m = n(n-1)$
- **Sparse graph: fewer edges**
  - ✓ Could be zero edges…

PATHS AND CYCLES

- A *path* is a sequence of vertices $P = (v_0, v_1, \ldots, v_k)$ such that, for $1 \le i \le k$, edge $(v_{i-1}, v_i) \in E$ (edge between v_{i-1}, v_i exists).

- Path P is *simple* if no vertex appears more than once in P.

$P_1 = (g, d, e, b, d, a, h)$ is not simple.



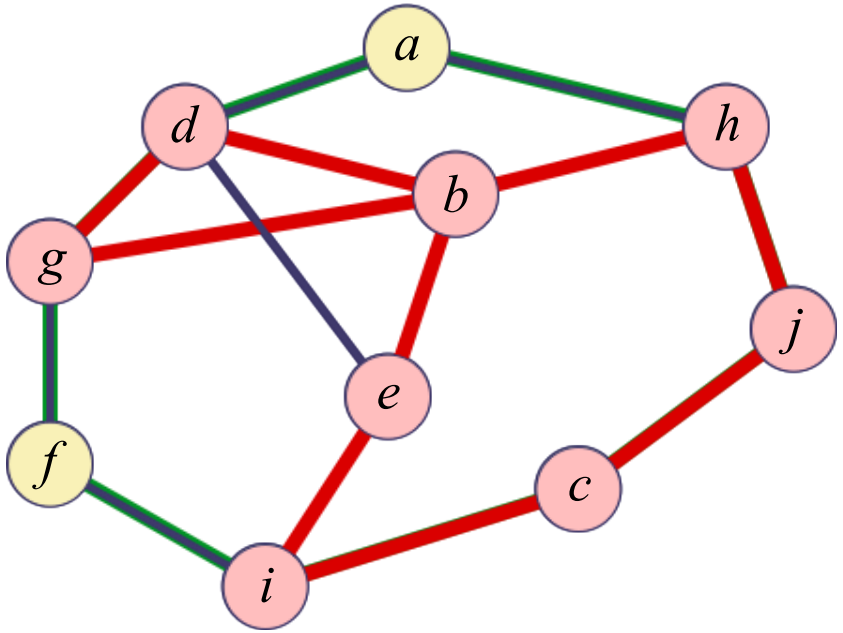
P_2 $P_2 = (f, i, c, j)$ *is simple.*

PATHS AND CYCLES

- A *cycle* is a sequence of vertices $C = (v_0, v_1, \ldots, v_{k-1})$ such that, for $0 \le i < k$, edge $(v_i, v_{(i+1) \bmod k}) \in E$.

- Cycle C is *simple* if the path $(v_0, v_1, \ldots, v_{k-1})$ is simple.

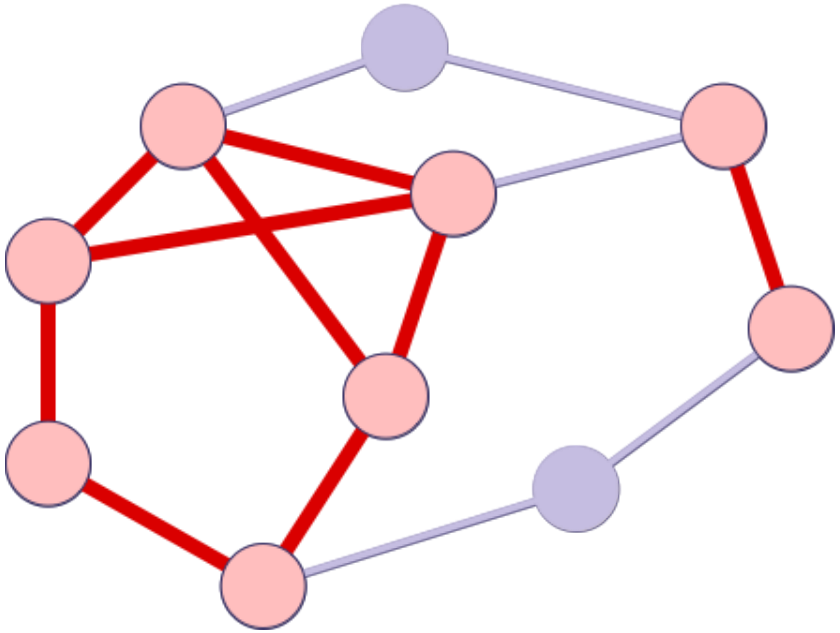
$C_1 = (a, h, j, c, i, f, g, d)$ is simple.



$C_2 = (g, d, b, h, j, c, i, e, b)$ is not simple.

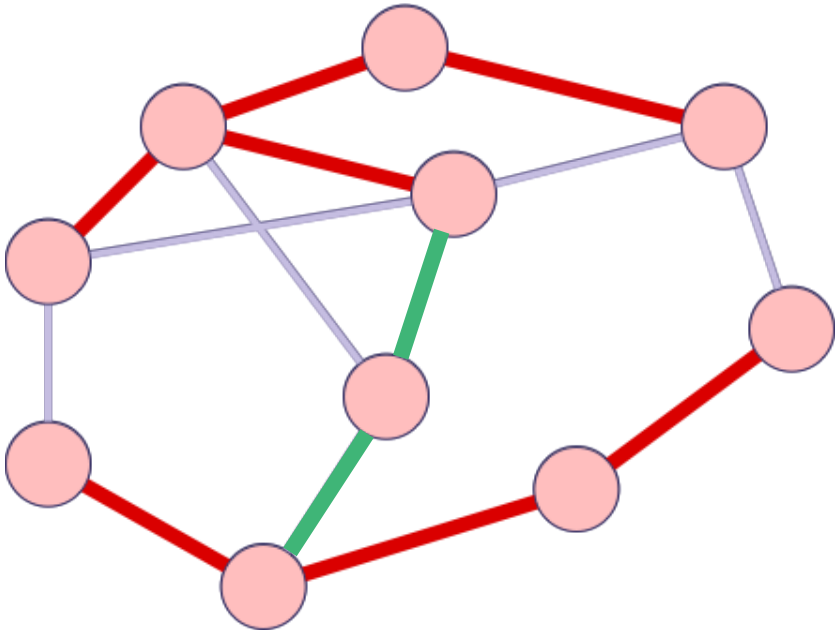
SUBGRAPHS

- A graph $H = (W, F)$ is a *subgraph* of a graph $G = (V, E)$
- if $W \subseteq V$ and $F \subseteq E$.



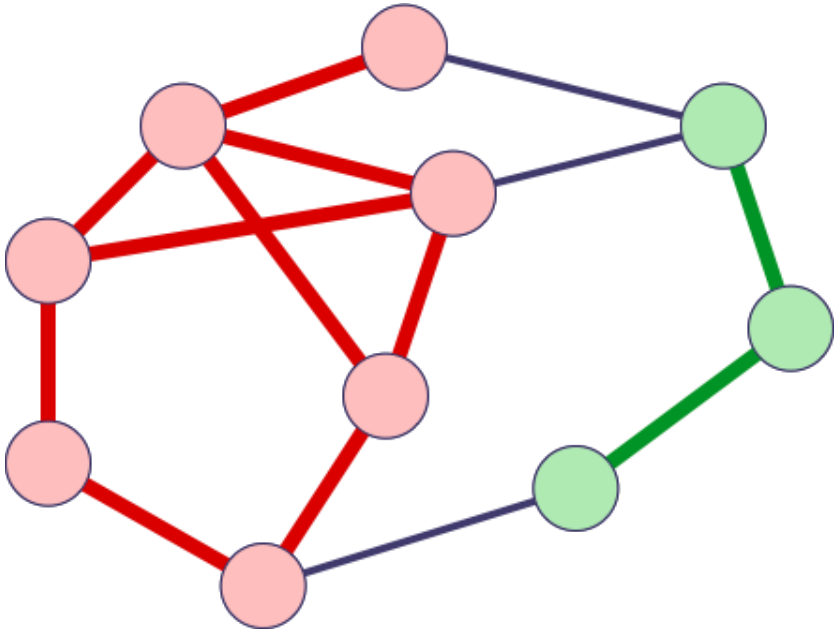
Spanning Graphs

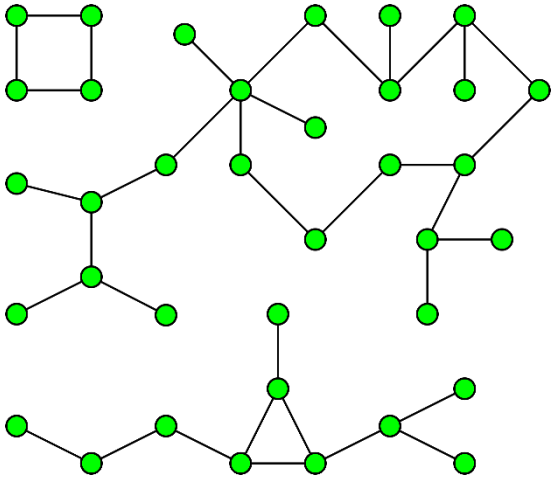
- A *spanning graph* of G
  is a subgraph of G that contains all vertices of G.



CONNECTIVITY : UNDIRECTED GRAPH

- A graph G is *connected* if there is a path between any two vertices in G.
- The *connected components* of a graph are its maximal connected subgraphs.





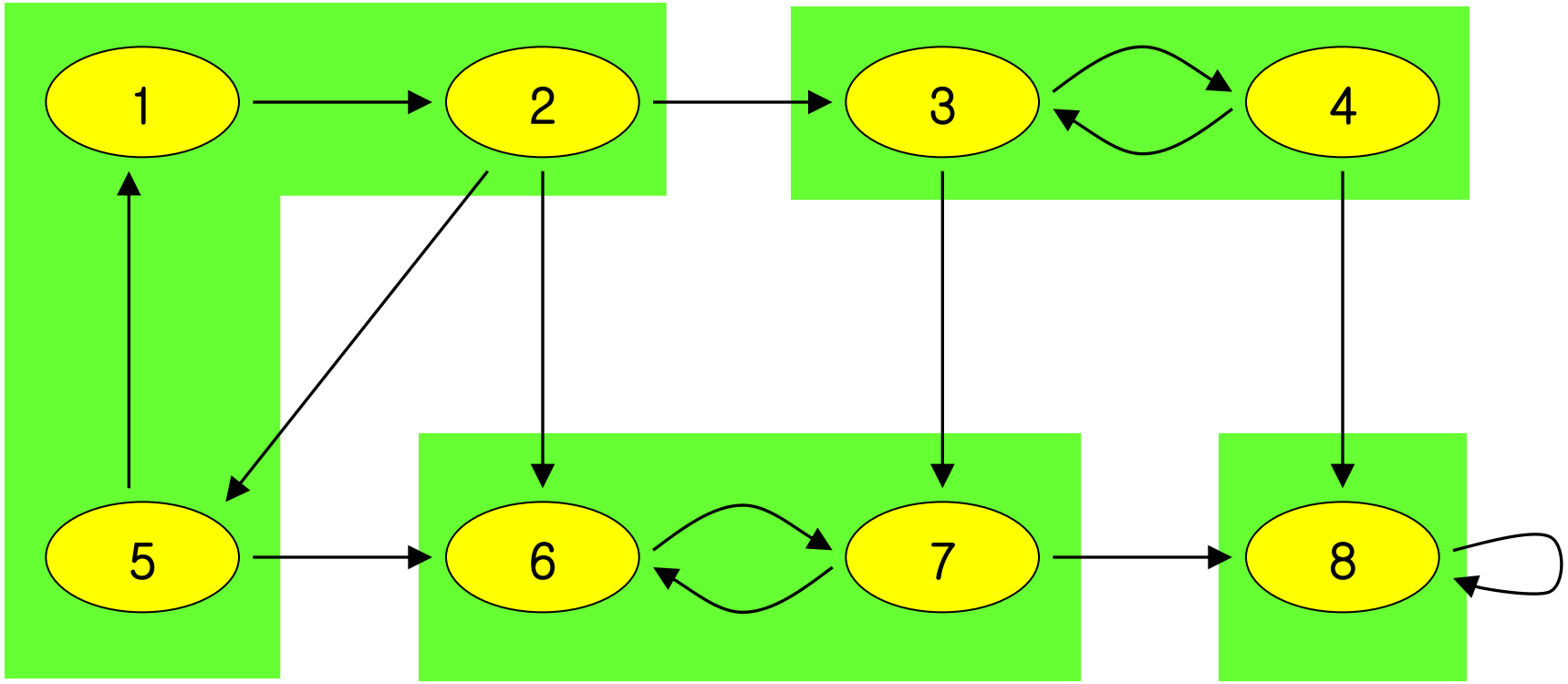
There are three connected components.

AND MORE TERMINOLOGY

- **A strongly connected <u>di</u>graph:**
  - ✓ direction affects this!
  - ✓ node u may be reachable from v, but not v from u
    - not strongly connected
  - ✓ <u>Strongly</u> connected means there exist both directions
- **Acyclic: no-cycles**
- **Directed acyclic graph: a DAG**

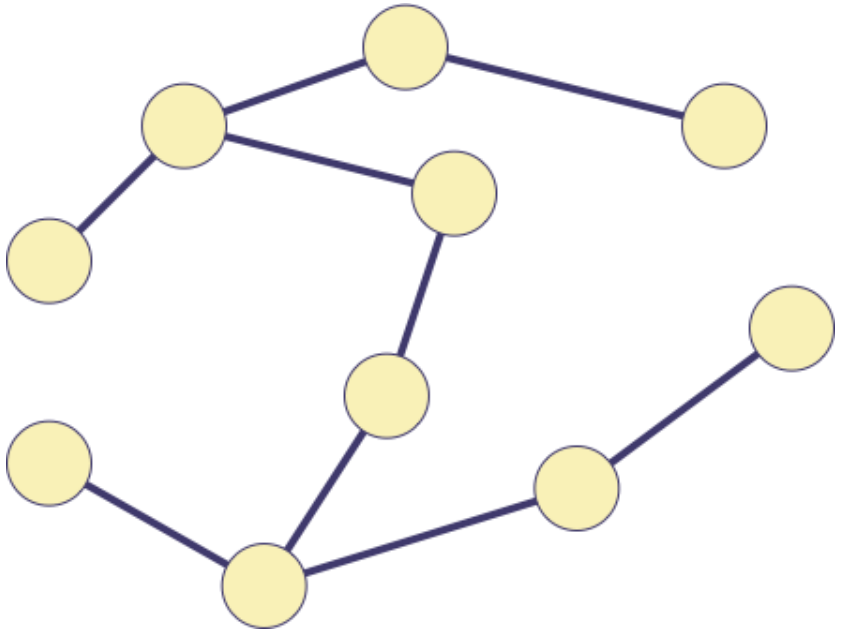
EXAMPLE

- **Green areas are the Strongly Connected Components.**



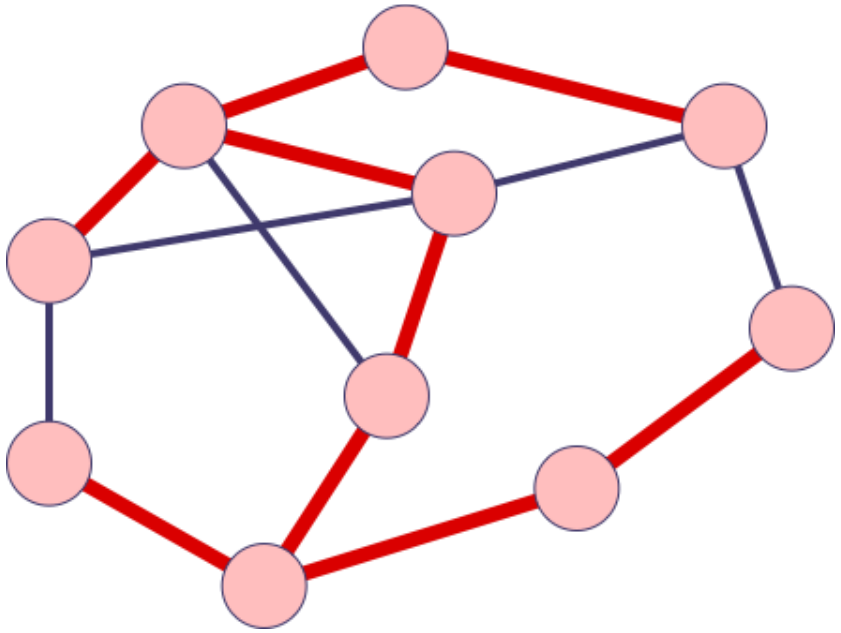
TREES

- A *tree* is a graph that is connected and contains no cycles.



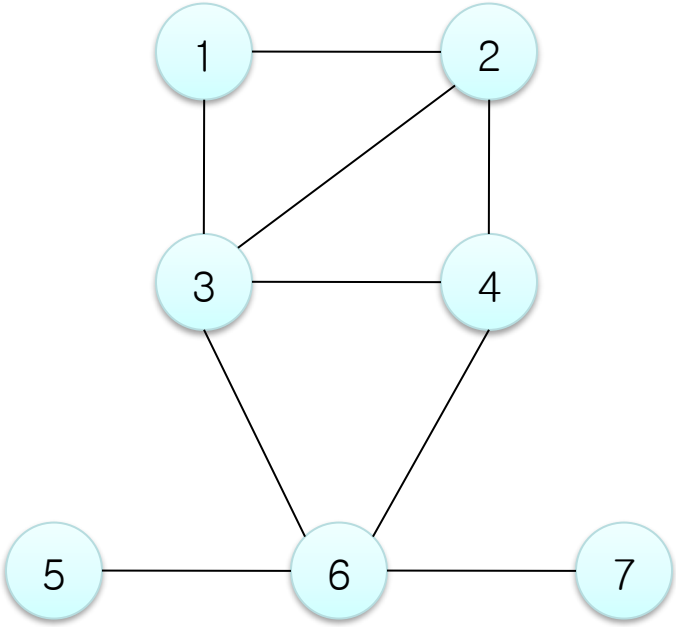
SPANNING TREES

- A *spanning tree* of a graph is a spanning graph that is a tree.



❖ Adjacency Matrix Representation

- ✓ Let $G = (V, E)$, $n = |V|, m = |E|$, $V = \{v_1, v_2, \dots, v_n\}$
- ✓ G can be represented by an $n \times n$ matrix



(a) An undirected graph

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(b) Its adjacency matrix

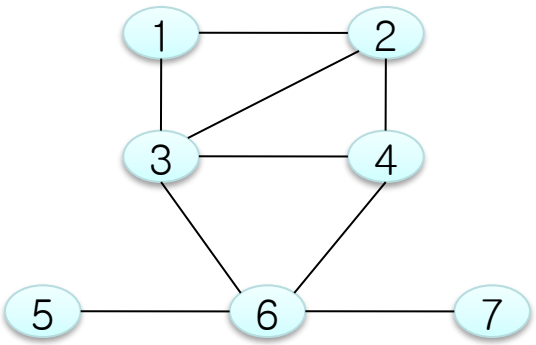
GRAPHS: ADJACENCY MATRIX

- **Q: How much storage does the adjacency matrix require?**
  - ✓ A: $O(V^2)$
- **Q: What is the minimum amount of storage needed by an adjacency matrix representation of an undirected graph with 4 vertices?**
  - ✓ A: 6 bits
    - Undirected graph → matrix is symmetric
    - No self-loops → don't need diagonal

Graphs: Adjacency Matrix

- **Preferred when a graph is dense.**
  - ✓ Usually too much storage for large graphs
  - ✓ But can be very efficient for small graphs
- **Most large interesting graphs are sparse**
  - ✓ For this reason, the adjacency list is often a more appropriate representation

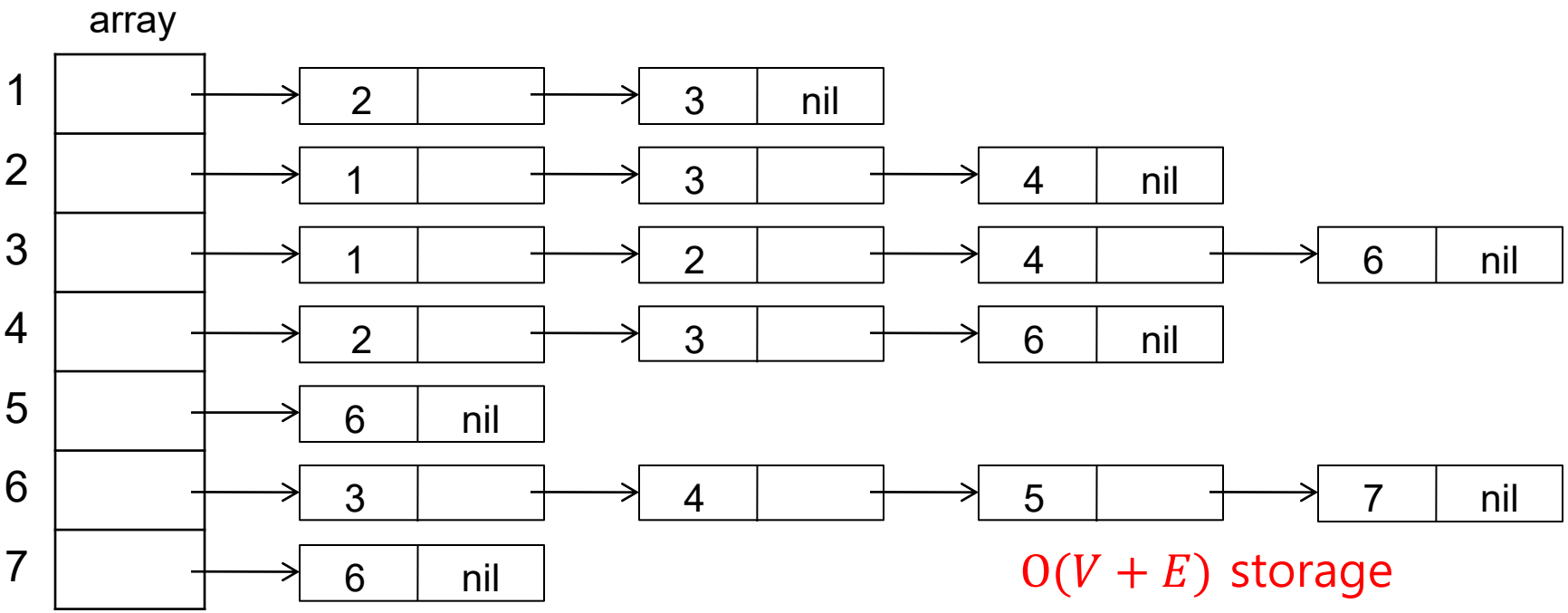
Array of Adjacency Lists Representation



(a) An undirected graph

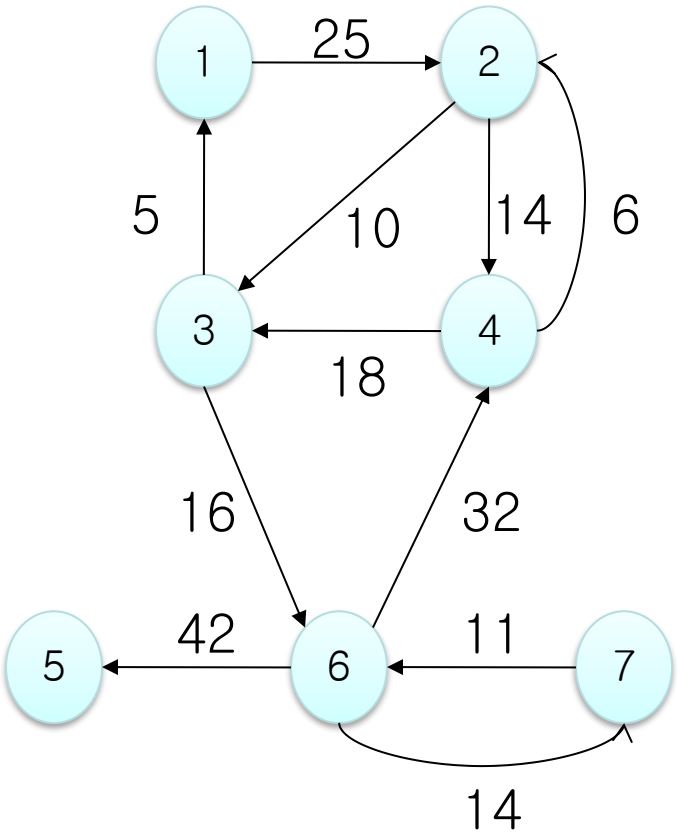
$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(b) Its adjacency matrix



$O(V + E)$ storage

ADJACENCY MATRIX FOR WEIGHT DIGRAPH

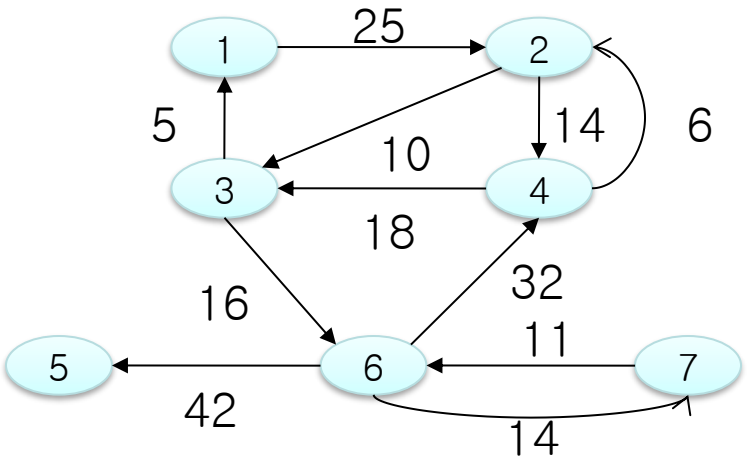


(a) A weighted digraph

$$\begin{pmatrix} 0 & 25.0 & \infty & \infty & \infty & \infty & \infty \\ \infty & 0 & 10.0 & 14.0 & \infty & \infty & \infty \\ 5 & \infty & 0 & \infty & \infty & 16.0 & \infty \\ \infty & 6.0 & 18.0 & 0 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & 32.0 & 42.0 & 0 & 14.0 \\ \infty & \infty & \infty & \infty & \infty & 11.0 & 0 \end{pmatrix}$$

(b) Its adjacency matrix

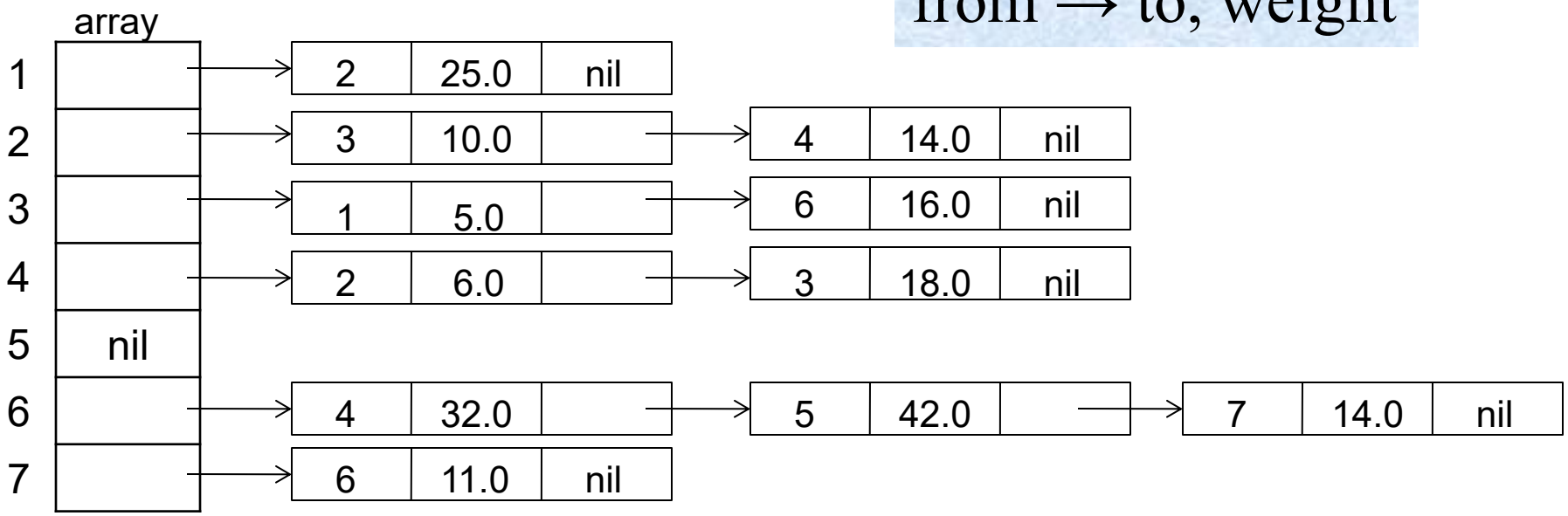
Array of Adjacency Lists Representation



(a) A weighted digraph

$$\begin{pmatrix} 0 & 25.0 & \infty & \infty & \infty & \infty & \infty \\ \infty & 0 & 10.0 & 14.0 & \infty & \infty & \infty \\ 5 & \infty & 0 & \infty & \infty & 16.0 & \infty \\ \infty & 6.0 & 18.0 & 0 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & 32.0 & 42.0 & 0 & 14.0 \\ \infty & \infty & \infty & \infty & \infty & 11.0 & 0 \end{pmatrix}$$

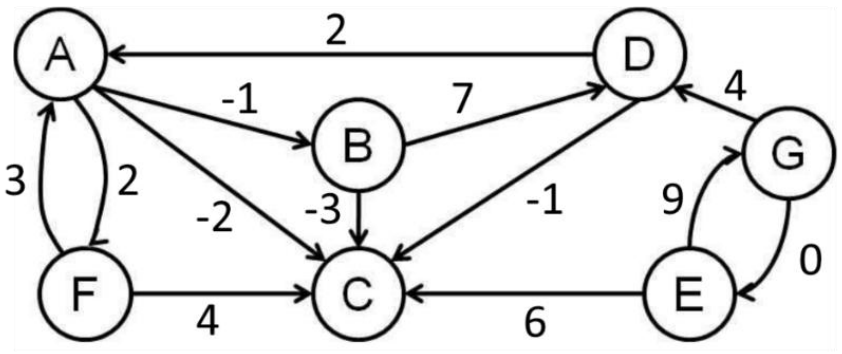
from → to, weight



ASSIGNMENT

❖ Assignment

- ✓ Given a graph,



- ✓ Fill the adjacency matrix below.

	A	B	C	D	E	F	G
A							
B							
C							
D							
E							
F							
G							

- ✓ Write(Draw) the array of adjacency list in alphabetical order.
- ✓ Write a report that includes your answers to the questions and upload your report to HDLMS.

Thank you!